

Object Constraint Language: a Definitive Guide

O artigo intitulado "Object Constraint Language (OCL): a Definitive Guide", Jordi Cabot e Martin Gogolla aborda o que poderíamos chamar de "crise de identidade" das linguagens gráficas: elas são ótimas para desenhar a estrutura, mas péssimas para lidar com detalhes minuciosos. Se o UML é o "esqueleto" visual de um sistema, o OCL é o sistema nervoso que garante que cada articulação se mova apenas como deveria. O texto deixa claro que, sem o OCL, um diagrama de classes é apenas a "ponta do iceberg", deixando questões críticas como se um cliente negativedo pode alugar um carro ou como calcular o preço de uma reserva totalmente à mercê da imaginação do desenvolvedor.

A essência do OCL, conforme descrita no guia, reside na sua natureza declarativa e livre de efeitos colaterais. Diferente de uma linguagem de programação comum, você não usa OCL para "fazer" algo, mas para "descrever" o que deve ser verdade. Ele é um suporte para definir invariantes (regras que nunca podem ser quebradas), contratos de operação (pré e pós-condições) e regras de derivação. O que torna o OCL robusto é o seu sistema de tipos, que vai desde os básicos atômicos (Integer, Real, String) até tipos de template complexos, como coleções e tuplas. O coração da linguagem bate forte no processamento dessas coleções, onde os autores explicam a lógica por trás dos quatro "cavaleiros" do OCL: Set, Bag, Sequence e OrderedSet.

A distinção entre essas coleções é pura lógica matemática. Um Set é insensível tanto à ordem quanto à frequência de inserção, enquanto uma Bag ignora a ordem mas mantém as duplicatas. Já a Sequence e o OrderedSet respeitam a ordem, mas diferem na forma como tratam a frequência. Para lidar com o mundo real onde nem tudo é perfeito o OCL utiliza uma lógica de três valores, introduzindo o valor null para representar estados indefinidos, como uma divisão por zero ou uma informação ausente no banco de dados.

Se você quer saber quem manda nas coleções, o artigo aponta para a operação iterate. Ela é descrita como a operação mais poderosa do OCL, funcionando como a "peça fundamental" a partir da qual todas as

outras operações como select, reject ou collect podem ser derivadas. A sintaxe do iterate é um pouco mais densa, exigindo uma variável de elemento, um acumulador e uma expressão de iteração, mas é ela que dá ao desenvolvedor o controle total sobre como transformar um conjunto de dados bruto em uma resposta refinada.

No campo das ferramentas, o cenário é de crescimento, mas com ressalvas. Temos parsers sólidos como o MDT/OCL (parte do ecossistema Eclipse) e o Dresden OCL, além de ambientes de validação como o USE (*UML-based Specification Environment*), que permitem criar "snapshots" do sistema para verificar se as regras de negócio fazem sentido na prática. Contudo, os autores admitem que a verificação formal de OCL é um desafio hercúleo: como o raciocínio sobre OCL é, em geral, indecidível, as ferramentas costumam focar em "verificação limitada" (*bounded verification*), explorando espaços de estados finitos para garantir que o sistema não entre em colapso lógico.

Encerrando, o artigo aponta caminhos necessários para a evolução da linguagem: a necessidade de modularização (importar bibliotecas de terceiros), melhorias na sintaxe para torná-la menos verbosa e, crucialmente, técnicas de raciocínio mais eficientes para lidar com modelos gigantescos, como os obtidos via engenharia reversa. Fica nítido que o OCL não é apenas um "puxadinho" do UML, mas sim o que torna o *Model-Driven Engineering* (MDE) tecnicamente viável e rigoroso.